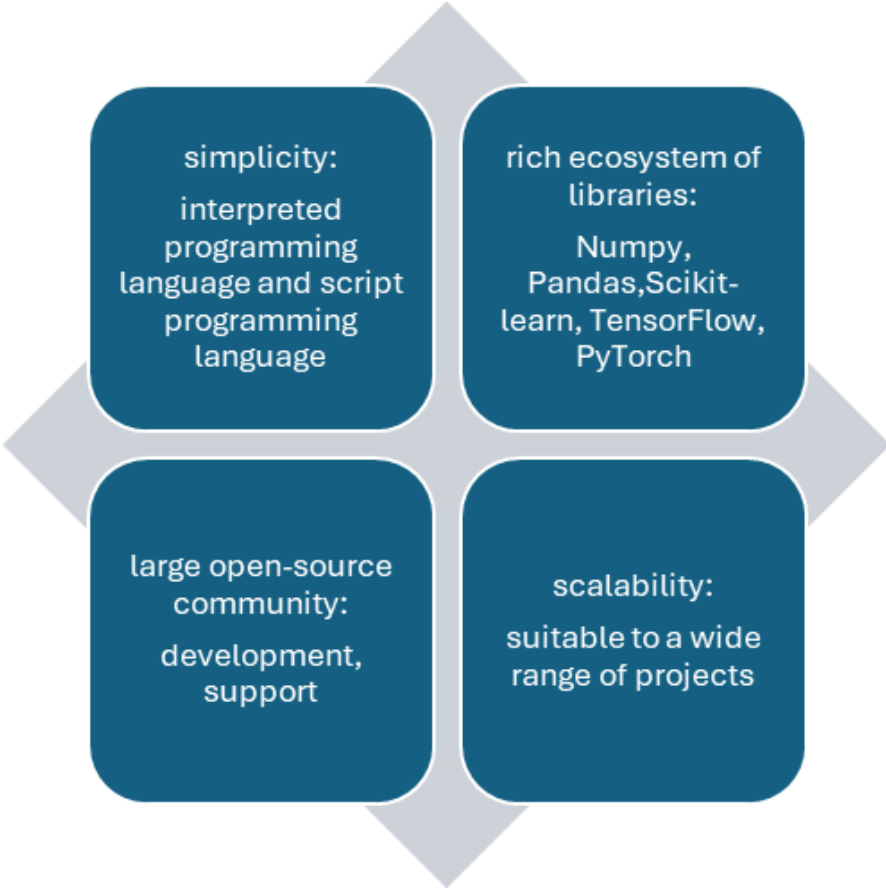
Chapter 1 Python Programming Language Review

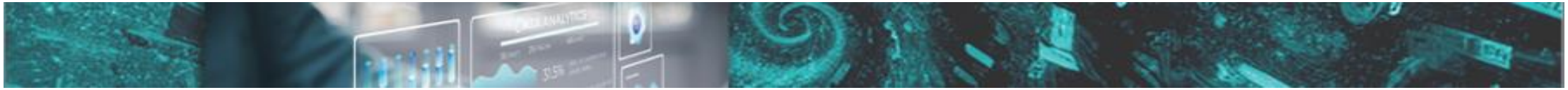
Data Analytics Using Python

Dr. Ziping Liu

Python Advantages

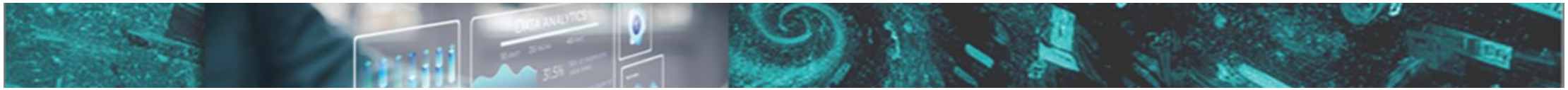
- Popular programming language data science and artificial intelligence
- offers the advantages of simplicity, a rich ecosystem of libraries, a large open-source community, and scalability
- Unlike compiled languages such as C++,Java
- is an interpreted language, doesn't need to be translated into machine code before running. code executed line by line by an interpreter at runtime
- has characteristics of a scripting language, being easy to learn and code, suitable for automating tasks and building prototypes.





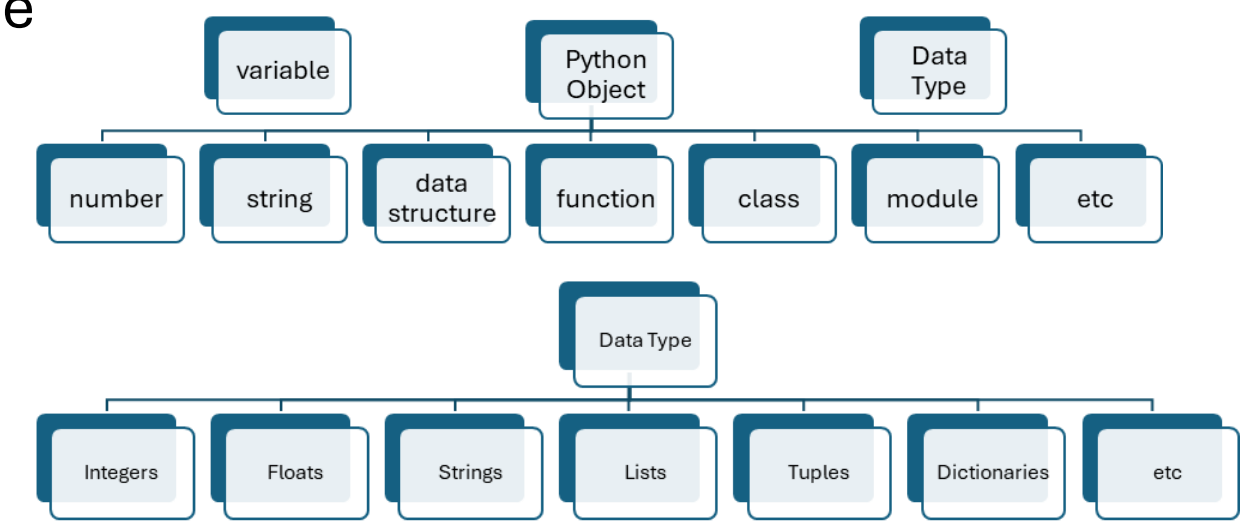
Getting Started with Python

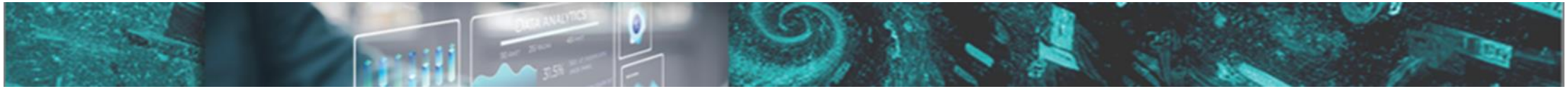
- Installation
  - Install Python
  - Install Anaconda
- Integrated development environments (IDEs)
  - PyCharm: A Python IDE for data science and web development.
  - Visual Studio Code: A code editor developed by Microsoft for every major programming language.
  - Spyder: A free and open-source scientific environment written in Python, for Python.
  - Jupyter Notebook: A web application for creating and sharing computational documents.
- Google Colab: web-based, no need to install Python and Jupyter Notebook locally



Python Basics

- Variables and Data Types
  - Variables are used to store data.
  - Python variables are not explicitly typed.
  - Python data types are associated with the objects that the variables reference



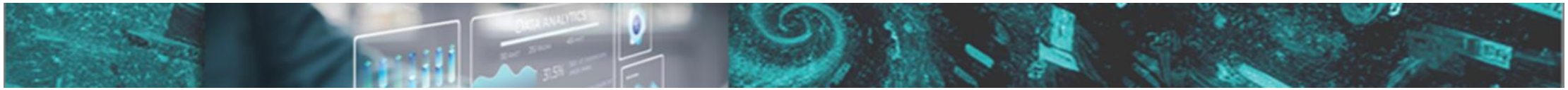


Python Basics

- Python numbers
  - include integers and floats
  - Integers: whole numbers without decimal points
  - Floats: numbers with decimal points

```
int_var = 10
float_var = 9.99999
print(f"int_var = {int_var}, float_var = {float_var:.2f}")
```

the running result is: `int_var = 10, float_var = 10.00`

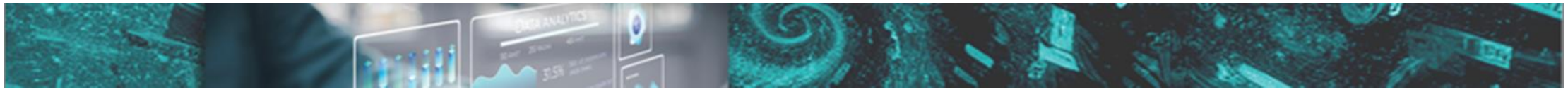


Python Basics

- Python Strings
  - textual data enclosed in single, double, or triple quotes
  - triple quotes allowing for multi-line strings or docstrings

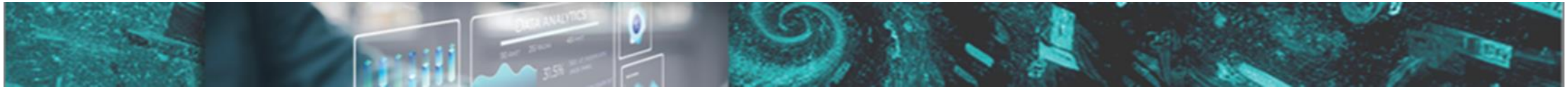
```
one_quote_var = 'hello, world'
two_quote_var = "hello, world"
three_quote_var = '''hello
                    world'''
print(f"one_quote_var = {one_quote_var}\ntwo_quote_var =
{two_quote_var}\nthree_quote_var = {three_quote_var}")
```

```
the running result is:
one_quote_var = hello, world
two_quote_var = hello, world
three_quote_var = hello
                  world
```



Python Basics

- Python objects
  - include collections of elements, such as Lists, Tuples, Dictionaries and Sets

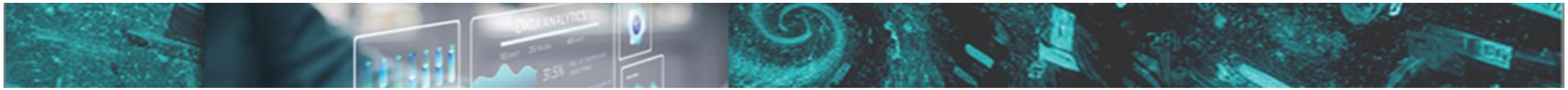


Python Basics

- Python Lists
  - ordered collections of elements and elements
  - can be any data types
  - can store heterogeneous collections of data
  - can be created with brackets, and elements are separated by commas
  - mutable

```
var_list = [10, "hello", 9.99, True, [1, 2, 3],
{"language": "Python"}]
print("before add a new item, var_list: ", var_list)
var_list.append(['new', 'item'])
print("after add a new item, var_list: ", var_list)
```

- Exercise: use other approaches to create var_list, update one item in the list, remove one item from the list

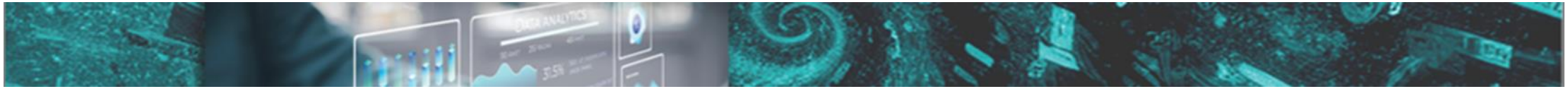


Python Basics

- Python tuples
  - ordered
  - Immutable: cannot add new items to them or remove existing ones
  - can store heterogeneous collections of data
  - can be created with parentheses (), and elements are separated by commas

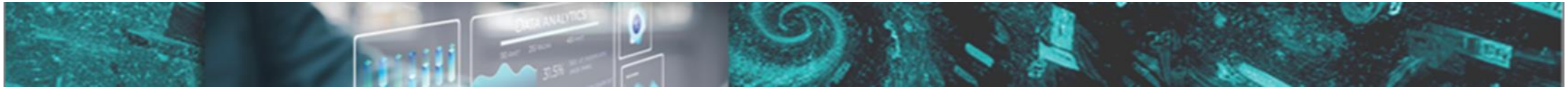
```
var_tuple = (10, "hello", 9.99, True, [1, 2, 3], {"language": "Python"})
print("var_tuple: ", var_tuple)
```

- Exercise: use other approaches to create var_tuple, output one of the items



Python Basics

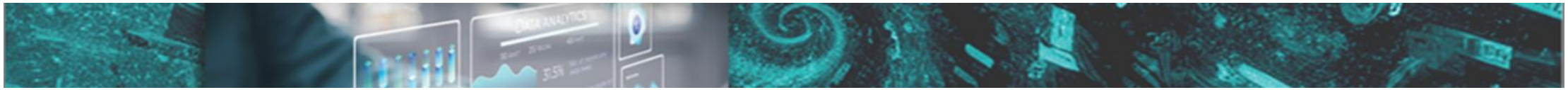
- Python Dictionaries
  - key-value pairs
    - keys are unique identifiers, mmutable and hashable
    - common data types for keys include strings, numbers, or tuples
    - values can be any data type - numbers, strings, lists, dictionaries, or even custom objects
    - values store the actual information associated with the key
  - do not preserve the order in which elements were added
  - mutable
  - can be created with curly braces {}, and elements are separated by commas. Colons : separate the keys from their values
- Exercise: use other approaches to create apple_stock, remove one key-value pair



Python Basics

- Python Dictionaries

```
# define a dictionary for apple stock
apple_stock = {"symbol": "AAPL", "shares": 100, "price": 150.25, "total_value": 15025.00}
# Accessing values by key
ticker = apple_stock["symbol"]
# Modifying a value
apple_stock["shares"] = 150
# Adding a new key-value pair
apple_stock["year"] = 2023
# get all keys as a list
print("all keys: ", list(apple_stock.keys()))
# get all values as a list
print("all values: ", list(apple_stock.values()))
# get all key-value pairs as tuples of list
print("apple stock: ", list(apple_stock.items()))
# If the key 'going up' exists in the dictionary appleStock, the get() method will return the associated value.
# However, if the key doesn't exist, the get() method will return the second argument, which is 'NaN' (Not a Number) in
this case.
print(apple_stock.get('going up', 'NaN'))
# Unlike get(), setdefault() modifies the dictionary if the key doesn't exist.
# the key 'going down' is missing, setdefault() will add it to the dictionary with the value 'NaN'.
apple_stock.setdefault('going down', 'NaN')
print("apple stock: ", list(apple_stock.items()))
```

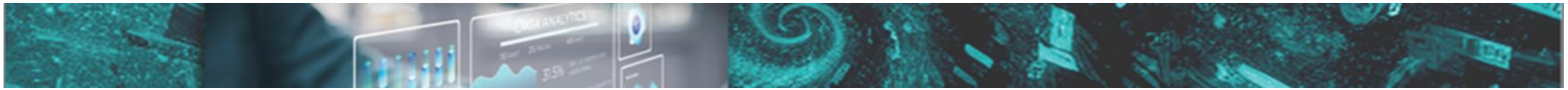


Python Basics

- Python Sets
  - unordered collections of unique elements
    - Elements themselves immutable and hashable
  - mutable, allow to add or remove elements
  - can be created with curly braces {}, and elements are separated by commas

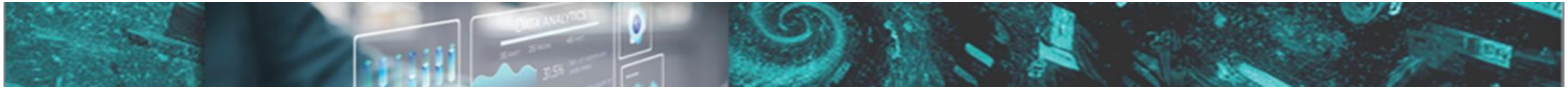
```
var_set = {10, 10, "hello", 9.99, True, (1, 2, 3)}

Print("var_set: ", var_set)
```

- Exercise: use other approaches to create var_set, update one item in the list, add a new item, remove one item from the set



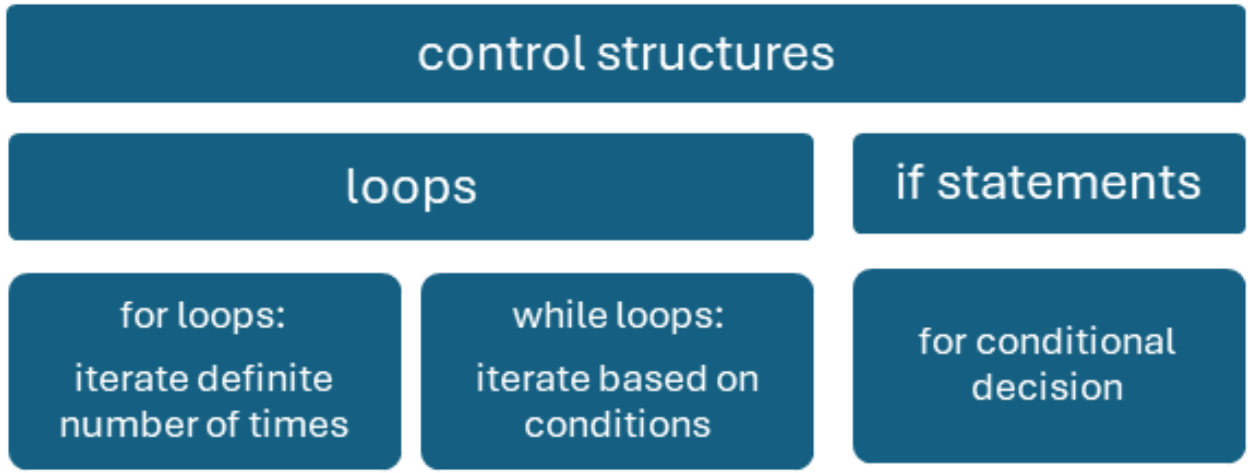
Python Basics

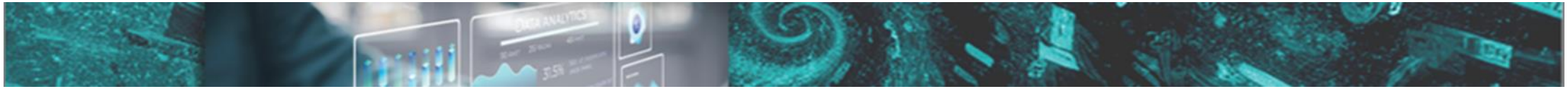
- Python Indentation
  - code blocks are defined by colon(:)
  - typically using 4 spaces or tabs
  - determines the scope of loops, conditionals, functions, classes, and other constructs



Python Basics

- Control Structures





Python Basics

- if statement: conditional execution, allowing code to run based on whether a specified condition is true or false

The usage for two-way conditions

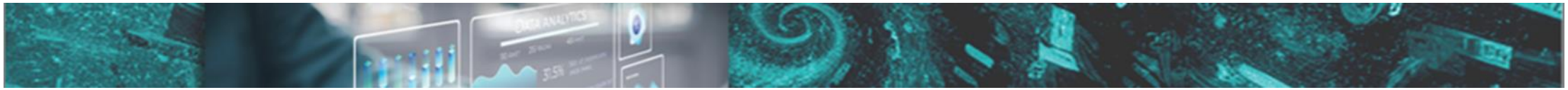
```
if condition:
  # code to execute if the condition is True
else:
  # code to execute if the condition is False
```

The usage for multi-way conditions

```
if condition1:
  # code to execute if condition1 is True
elif condition2:
  # code to execute if condition1 is False and condition2 is True
elif condition3:
  # code to execute if condition1 and condition2 are False and
condition3 is True
else:
  # code to execute if all conditions are False (optional)
```

The usage for nested if

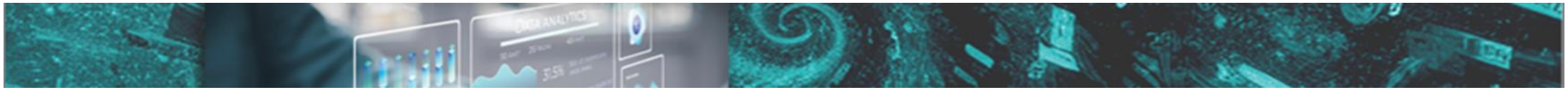
```
if condition1:
  # code to execute if condition1 is True
  if condition2:
    # code to execute if condition1 is True and condition2 is also
True (nested if)
  else:
    # code to execute if condition1 is True but condition2 is False
else:
  # code to execute if condition1 is False
```



Python Basics

- If statement

```
investment = 1000
current_price = 1155
purchase_price = 1100

if purchase_price == 0:
    roi = null
else:
    roi = (current_price - purchase_price) / purchase_price * 100

earned = investment * roi / 100

if roi > 10:
    print(f"Your investment has a strong return of {roi:.2f}% and you earn {earned: .2f}!")
elif roi > 0:
    print(f"Your investment has a positive return of {roi:.2f}% and you earn {earned: .2f}")
elif roi == 0:
    print(f"Your investment has broken even and you earn {earned: .2f}.")
else:
    print(f"Your investment has a negative return of {abs(roi):.2f}% and you earn {earned: .2f}")
```



Python Basics

- loop: repetitive tasks and/or collections of data

for/definite loop: iterate over a sequence (such as a list, tuple, string, or range):

```
for element in sequence:
    # Code block to be executed
```

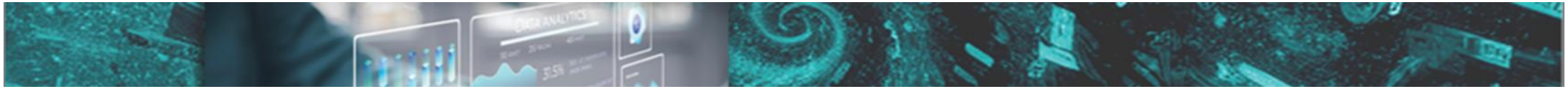
While/Indefinite loop: execute a block of code as long as a condition is true

```
while condition:
    # Code block to be executed
```

initializaton: setting up a loop variable to track the number of iterations of the loop

condition: an expression on loop variable to determine if the loop continues. The loop continutes if the condition is true

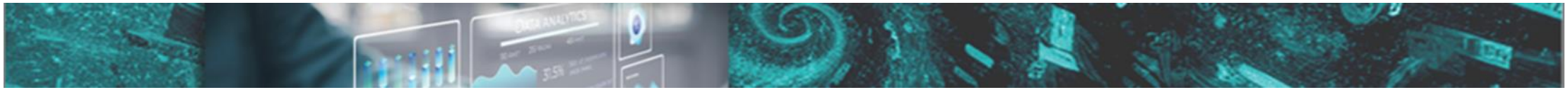
loop body: indented block of code that will be executed repeatedly. At the end of the loop body, updates on the loop variable so that loop will terminate eventually.



Python Basics

- for statement

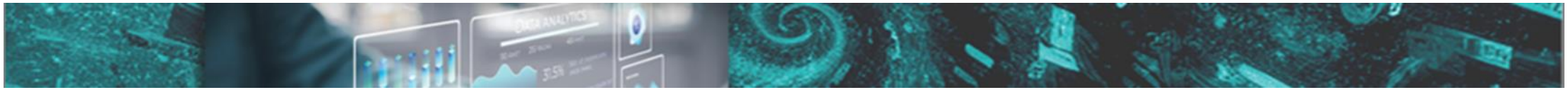
```
import random
stock_data = {
  "AAPL": round(150.23 + random.random() * 10, 2),
  "TSLA": round(900.00 + random.random() * 50, 2),
  "GOOG": round(2800.00 - random.random() * 100, 2)
}

for symbol, price in stock_data.items():
  print(f"Stock: {symbol}, Price: ${price}")
```



Python Basics

- while statement

```
aapl_price = 159.23
guess = 180
guess_times = 1 # A counter is initialized to keep track of the number of guesses

while guess != aapl_price:
    guess = float(input("Guess apple's stock price: "))   # Prompts the user to guess the stock price
and converts the input to a float
    if abs(guess - aapl_price) <= 10 and guess_times < 3: # abs(): find absolute value of an expression
        print("Congratulations, you guessed the price!")
        break                                             # exits the loop
    elif abs(guess - aapl_price) > 10 and guess_times < 3:
        print("Try again!")
    else:
        print("Run out of guessing times!")
        break                                             # exits the loop
    guess_times +=1                                       # increments the guess counter after each
guess
```

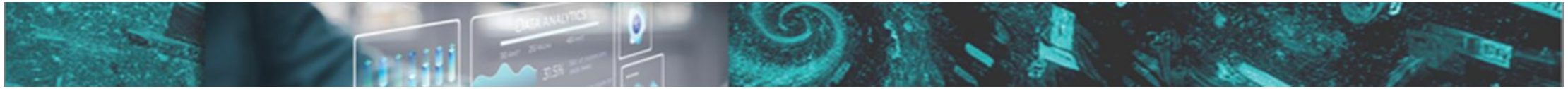


Python Basics

- Functions: reusable blocks of code designed to perform specific tasks

function definition:

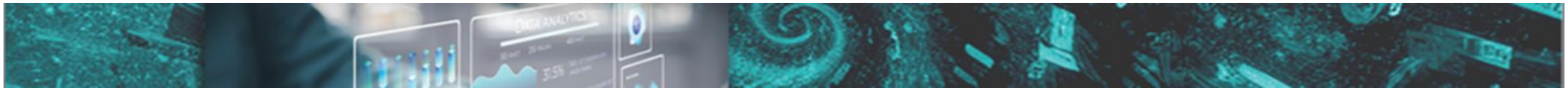
```
def function_name(parameters): # function header
    """Function docstring (optional)"""
    # Function body (indented code block)
return output_value (optional)
```



Python Basics

- function

```
import yfinance as yf  # import yfinance library for stock data

def get_stock_data(symbol): # define function header
    try:
        stock_data = yf.download(symbol, period="1wk", interval="1d")  # download 1-week daily data from Yahoo Finance
    except:
        print(f"Error: Could not retrieve data for symbol {symbol}.")
        return None

    current_price = stock_data["Close"].iloc[-1]   # get latest closing price
    current_volume = stock_data["Volume"].iloc[-1] # get latest volume
    return {"Stock ticker": symbol, "Current Price": current_price, "Current Volume": current_volume}  # return a
dictionary

# passing google, apple and tesla's stock tickers
stock_symbol = ["GOOG", "AAPL", "TSLA"]
for ticker in stock_symbol:
    data = get_stock_data(ticker)  # data is a dictionary returned from the get_stock_data() function
    if data:
        for key, value in data.items():
            print(f"{key}: {value}")
```

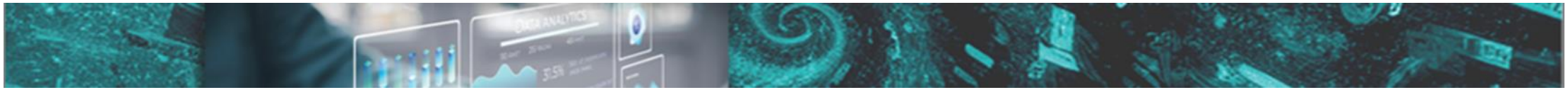


More on Python

- try-except block: prevent potential program crashes

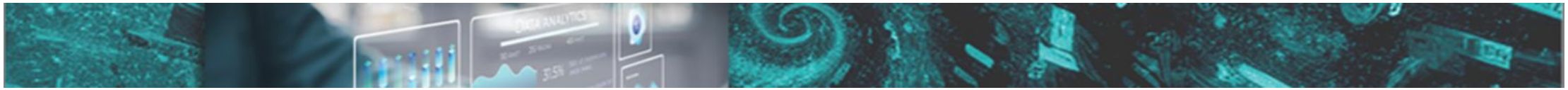
try-except block definition:

```
try:
    # Code that might raise an exception
except [ExceptionType]:
    # Code that continues to run in case of the exception
[except (ExceptionType1, ExceptionType2) as e:
    # Code that continues to run in case one of the exceptions]
…
[else:
    # Code that continues to run if no exception occurs
finally:
    # Code that always runs, regardless of exceptions]
```



More on Python

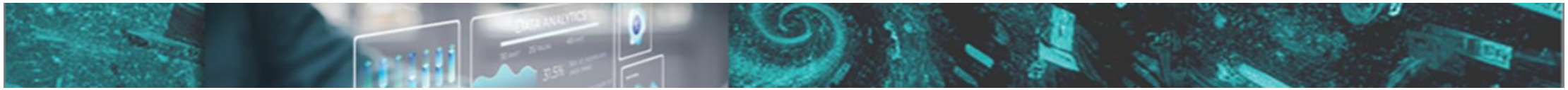
- Indexing and Slicing
  - access elements within sequences
  - indexing: extract a single element using its position
    - index starting from 0 (zero-based indexing)
    - use square brackets[] with the index within them to access an element
    - allows negative indexing to access elements from the end of the sequence: -1 refers to the last element, -2 to the second-last, and so on
  - Slicing: extract a sub-sequence
    - use square brackets [] with a start index, an end index, and an optional step (separated by colons): [start:end:step]
    - The end index in slicing is exclusive
    - can omit the start or end index in slicing to use the default values. If the step is omitted, the default step is 1



More on Python

- indexing and slicing

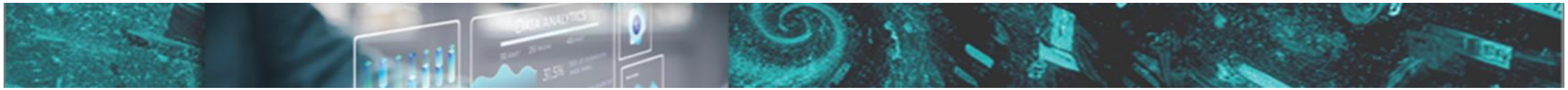
```
var_list = [10, "hello", 9.99, True, [1, 2, 3], {"language": "Python"}]
# access first item
print("first item: ", var_list[0])
# access last item
print("last item: ", var_list[-1])
# access the second to the fourth items
print("the second to the fourth items: ", var_list[1: 4])
# use slice assignment to append a list at back or at front, or remove part of a list
var_list[len(var_list):]=['X', 'Y', 'Z']
print("use slice to append at back: ", var_list)
var_list[:0] = ['A', 'B', 'C']
print("use slice to append at front: ", var_list)
var_list[9:-1]=[]
print("use slice to remove: ", var_list)
# use negative step for reversing
print("after reversing: ", var_list[::-1])
```



More on Python

- zip( ) built-in function
  - creates an iterator that aggregates elements from multiple sequences, and pair them together

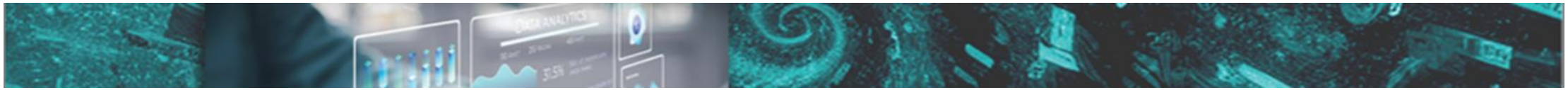
```
tickers = ["GOOG", "TSLA"]
prices = [172, 175]

# Combine tickers and prices into a list of tuples
stocks = list(zip(tickers, prices))  # zip returns an iterator, convert to
list
print(stocks)
```



More on Python

- * and ** as function arguments/parameters
  - * unpacking arguments from iterables (like lists, tuples) into individual arguments for the function

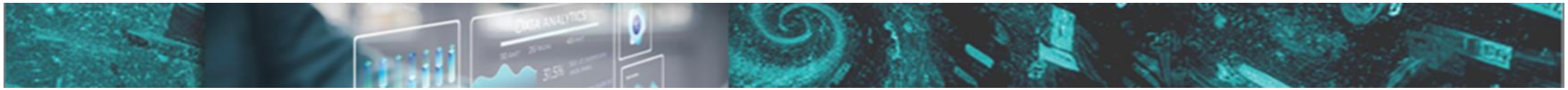
```
# Unpacking Arguments when used for arguments
def do_unpack(ticker, price):
    print(f"Storck Ticker: {ticker}, Price: {price}")

# Call do_unpack with each ticker-price tuple
for stock in stocks:
    do_unpack(*stock)  # Unpack the tuple into ticker and price arguments
```



More on Python

- * and ** as function arguments/parameters
  - * catching variable number of arguments: capture any number of arguments passed to the function as a tuple

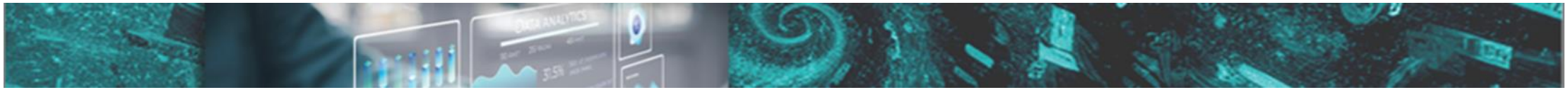
```
# Catching variable number of arguments when used for parameters
def sum_all(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

result = sum_all(1, 2, 3, 4, 5)
print(result)
```



More on Python

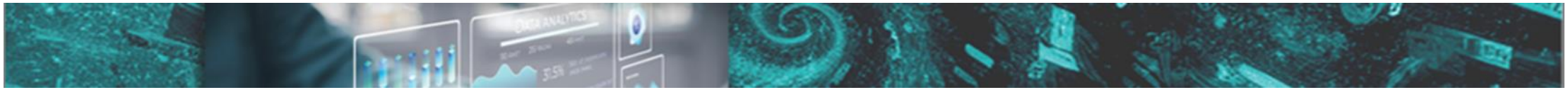
- ** syntax
  - can be used both in function parameters to accept keyword arguments and in keyword argument unpacking to pass arguments

```
def unpack_dictionary(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

# Creating a dictionary
stock_dict = dict(zip(tickers, prices))

# Passing the dictionary to the function using **
# **stock_dict unpacks the key-value pairs of the dictionary, and each pair
becomes a separate keyword argument to the function
unpack_dictionary(**stock_dict)
```



More on Python

- Python Comprehensions and Generator Expression
  - comprehension: build a sequence data structure, such as a list, set, or dictionary, using existing sequences
  - for tuple comprehension: use the tuple() constructor in combination with a generator expression
  - generator:
    - a special type of iterator that supports iteration over a sequence of elements
    - doesn’t store the entire sequence in memory but generates one element at a time on demand
    - memory-efficient and capable of handling even infinite sequences



More on Python

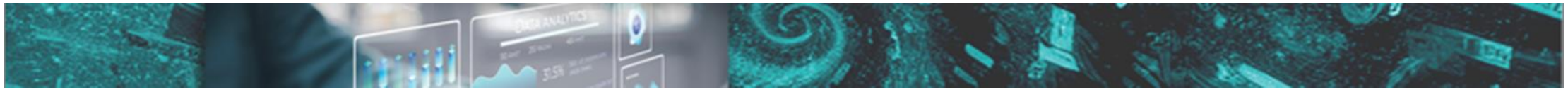
- Python Comprehensions and Generator Expression

Syntax to create a list comprehension:
```
[expression for val in collection if condition]
```

Syntax to create a set comprehension:
```
{expression for val in collection if condition}
```

Syntax to create a dictionary comprehension:
```
{key_expression : value_expression for val in collection if condition}
```

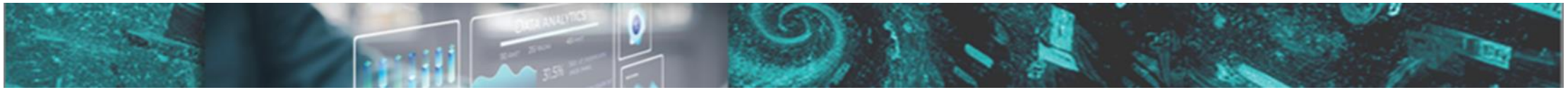
Syntax to create a generator expression:
```
(expression for val in collection if condition)
```



More on Python

- Python Comprehensions and Generator Expression

```
var_list = ['A', 'B', 'C', 10, 'hello', 'hello', 'hello', 9.99, True, [1, 2, 3], {'language': 'Python'}, 'X', 'Y',
'Z']
# build a list comprehension from an existing list's string elements
comp_list = [var for var in var_list if isinstance(var, str)]
print(comp_list)
# build a set comprehension from an existing list's string elements
comp_set = {var for var in var_list if isinstance(var, str)}
print(comp_set)
# build a dictionary comprehension from two existing lists
stock_symbol = ["GOOG", "AAPL", "TSLA"]
stock_price = [172, 190, 175]
stocks_share = {symbol:price for symbol, price in zip(stock_symbol, stock_price)}
print(stocks_share)
# build a generator comprehension from an existing list's string elements
comp_generator = (var for var in var_list if isinstance(var, str))
print(comp_generator)
for var in comp_generator:
    print(var, end = ' ')
print()
# build a tuple using generator comprehension
comp_to_tuple = tuple(var for var in var_list if isinstance(var, str))
print(comp_to_tuple)
```



More on Python

- Lambda Expressions
  - functions: have a header containing the function name and a list of parameters
  - lambda expressions are simple, anonymous functions that can be defined and used inline

Syntax for lambda expressions:
`lambda parameter1, parameter2, . . .: expression`

```
# two lambda expressions are used to do calculations between feet and meter
conversion
width = {'Feet to Meter': lambda feet: 0.305 * feet,
         'Meter to Feet': lambda meet: 3.281 * meter}
print(width['Feet to Meter'](3.9))
```